Robust Training of Neural Networks at Arbitrary Precision and Sparsity

C. Ye, G. Chu, Y. Liu, Y. Zhang, L. Lew, L. Zhang, M. Sandler, A. Howard
Google DeepMind • ICLR 2026

Presented by

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Why quantization, and the forward pass during training

Why quantization. Modern neural networks are huge: a language model has billions of weights, each stored as a 32-bit or 16-bit floating-point number. This makes models too big to fit on a phone, slower to run, and costly in energy. Quantization stores each weight with far fewer bits: 4, 2 or 1 bit instead of 16 or 32.

Training with quantization. During training the network keeps the float weights x and updates them by gradient descent. We could compute with x, but we deliberately use the quantized version $\hat{x}$ in the forward pass, so the network learns to be robust to the rounding it will face at inference. After training, x is discarded; only the low-bit integers q (plus one s_f, b_f per block) are kept.

The forward pass during training.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

- $x \in \mathbb{R}^N$: float block, kept and updated during training.
- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$: rescales x onto the k-bit integer grid; s_f = grid step in real units, b_f = real value of integer 0 ($b_f = 0$ in the linear case).
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the integers, with rounding error $|\delta_i| \le \frac{1}{2}$.
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$: x as seen after quantization; this is what the next layer uses.

The problem: the true gradient is zero

Chain rule through the quantizer. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^\top \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel: the local gradient is exactly the slope of round.

Round is a staircase.

$$\frac{d}{du}\,\mathrm{round}(u) = 0 \quad \text{for } u \notin \mathbb{Z} + \tfrac{1}{2}, \qquad \text{undefined at } u \in \mathbb{Z} + \tfrac{1}{2}.$$

Hence $\dfrac{\partial q}{\partial f(x)} = 0$ almost everywhere, so

$$\nabla_x L = 0.$$

Consequence. A small change in x does not change q, so the loss gives x no direction to move: x and every layer before it never update. Training cannot start.

The standard fix: Straight-Through Estimator (STE)

Idea. Forward pass: round exactly as before. Backward pass: pass the gradient straight through the rounding, i.e. replace the true slope of round (zero) by 1. This is a chosen rule, not a derived derivative.

Decomposition. Write the rounded value as the smooth part plus the rounding error:

$$q = \text{round}\big(f(x)\big) = f(x) + \delta, \qquad \delta = \text{stopgrad}\Big(\text{round}\big(f(x)\big) - f(x)\Big),$$

where stopgrad means: same value in the forward pass, treated as a constant (zero derivative) in the backward pass.

Forward. $f(x) + \delta = \text{round}\big(f(x)\big)$ exactly, so the network still sees quantized values.

Backward. δ is a constant, so

$$\frac{\partial q}{\partial f(x)} = \frac{\partial f(x)}{\partial f(x)} + \underbrace{\frac{\partial \delta}{\partial f(x)}}_{=0} = I.$$

Substituting into the chain rule of the previous slide,

$$\frac{\partial \hat{x}}{\partial x} = s_f \cdot I \cdot \frac{1}{s_f} = I, \qquad \text{hence} \qquad \nabla_x L = \nabla_{\hat{x}} L.$$

Effect. The gradient reaching x is no longer zero, so x and earlier layers update and training works.

Reference: Y. Bengio, N. Léonard, A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

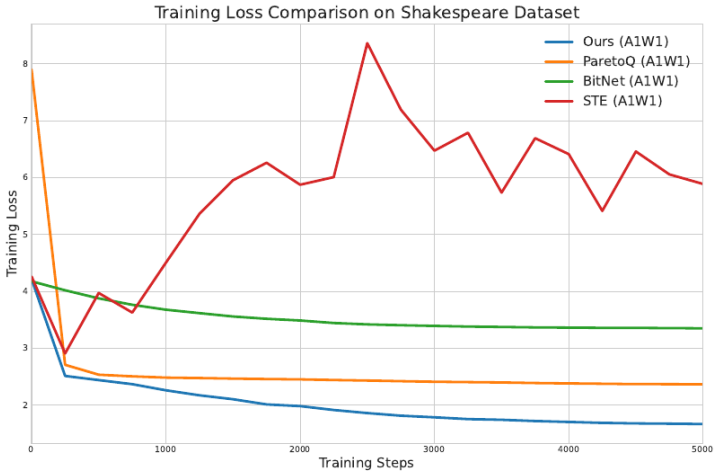
Why STE fails: the blind spot

Forward and backward disagree.

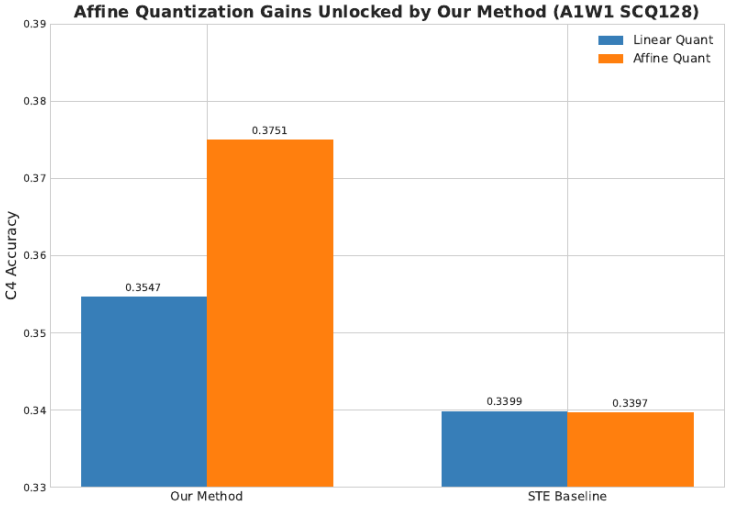
$$\text{forward:} \quad \hat{x} = x + s_f\,\delta, \qquad\qquad \text{backward:} \quad \frac{\partial \hat{x}}{\partial x} = I.$$

- The forward pass uses the rounded values: every entry is shifted by its rounding error $s_f \delta_i$, which changes the output and the loss.
- The local Jacobian is I: exactly what it would be with no quantization at all ($\hat{x} = x$). δ does not appear in it.
- So earlier layers get no signal about how rounding hurts the loss, and cannot learn to be robust to it. The paper calls δ a "ghost of departed quantities" (Berkeley, 1754).
- At 8 bits $s_f \delta$ is tiny and ignoring it is harmless. At 1–2 bits it is as large as the values themselves: the gradient no longer matches what the forward pass did, and training becomes unstable (STE diverges at A1W1, next slide, left).
- With affine quantization, STE barely learns the offset: no gain over linear (next slide, right).

STE at 1 bit: results from the paper



Training loss, 1-bit weights and activations (A1W1), small GPT on Shakespeare. STE (red) diverges; the paper's method (blue) converges.



Accuracy at A1W1, linear vs. affine quantization. Under STE affine gives no gain; with the paper's method it does.

Source: Ye et al., "Robust Training of Neural Networks at Arbitrary Precision and Sparsity," ICLR 2026, Fig. 1.

The fix: ridge regression dequantization

Why it works: the Jacobian sees δ

Role of λ

Sparsity as quantization

Theorem 1

Results from the paper

Toy implementation

Model. A small character-level GPT (decoder-only transformer): 4 blocks, 4 attention heads, embedding size 128, context of 64 characters, 0.82M parameters. Built following Karpathy's "Let's build GPT: from scratch" and nanoGPT, the same family as the paper's nanoGPT Shakespeare experiment.

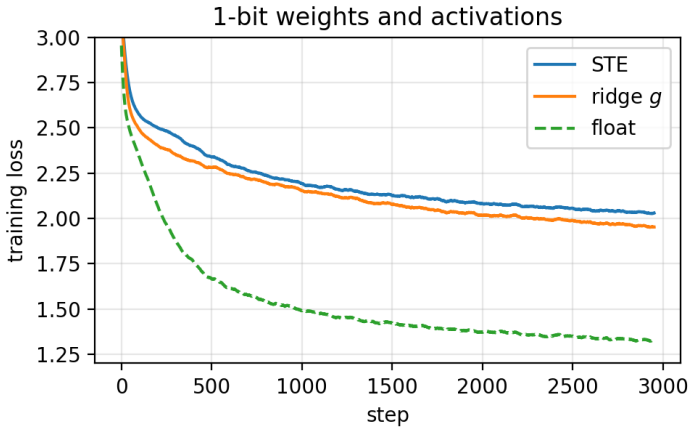
Data. Tiny Shakespeare: 1.1M characters, 65-character vocabulary, first 90% for training, last 10% for validation. Task: predict the next character.

Quantization. All linear layers inside the transformer blocks, weights and input activations, k bits (AkWk), affine, one s_f, b_f per row. Embeddings and the output layer stay full precision.

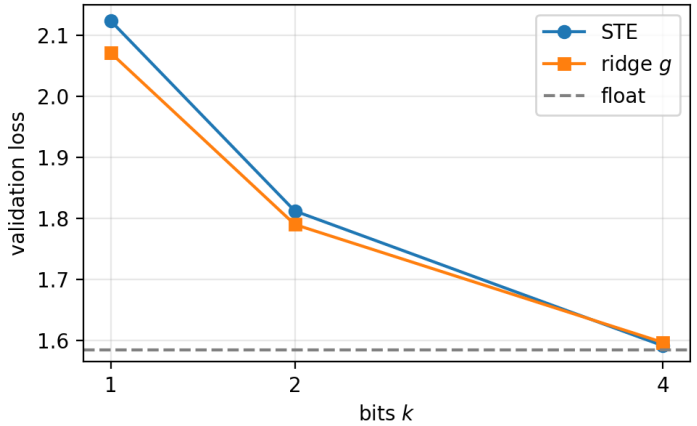
Runs. Same seed, AdamW (lr 10^{-3}), batch 64, 3000 steps; only the dequantization step differs. Float (no quantization, baseline); STE, $\hat{x} = s_f\, q + b_f \mathbf{1}$; ridge, $g(q) = s_g\, q + b_g \mathbf{1}$ with $\lambda = 0.01$. STE and ridge at $k = 1, 2, 4$.

References: A. Karpathy, "Let's build GPT: from scratch, in code, spelled out" (2023) and nanoGPT, `github.com/karpathy/nanoGPT`; Tiny Shakespeare from `github.com/karpathy/char-rnn`.

Toy implementation: results



Training loss at A1W1 (smoothed over 50 steps).



Final validation loss after 3000 steps.

Gradient check (1 bit, block of 64). Two inputs with the same q, rounding errors δ differing by up to 0.106, same upstream gradient:

	change in δ	change in gradient reaching x
STE	0.106	0.0000
ridge	0.106	0.0144

Size. Float32 model 3.19 MB; at 1 bit 250 KB (12.7× smaller).

Limitations and how to improve

Takeaway